Word Level Sign Language Recognition using MediaPipe and LSTM-GRU Network

1st Kumar Navendu
Department of ECE
Malaviay National Institute of Technology Jaipur
Jaipur, India
2022peb5150@mnit.ac.in, 0009-0007-7106-1870

2nd Vineet Sahula
Department of ECE
Malaviay National Institute of Technology Jaipur
Jaipur, India
vsahula.ece@mnit.ac.in, 0000-0001-9431-4518

Abstract

Indian Sign Language (ISL) is a crucial attribute to carrying out smooth and effective communication between the deaf and the hard-of-hearing community and those that can hear clearly. In this paper, we propose a new approach for word-level Indian Sign Language recognition using the MediaPipe framework, which is aided by the Long-Short Term Memory (LSTM) and Gated Recurrent Unit (GRU) network. Our approach uses robust hand and pose tracking features of MediaPipe to infer precise and comprehensive feature representations of sign gestures. These features are then fed into a hybrid LSTM-GRU network designed to capture Temporal dependencies, hence improving the accuracy of recognition.

In this paper, we test the proposed system using a varied dataset, INCLUDE [1], [10], for ISL signs, to show the system's capacity to deal with variation across signing styles and speeds. Experimental results indicate our approach outperforms current methods by large margins in recognition accuracy and processing system speed for the technique to work practically, ensuring real-time processing to ensure fast feedback to the user.

Besides, the integration of MediaPipe real-time tracking with state-of-the-art sequential modelling capabilities of LSTM-GRU networks is also very promising for the real-time application of ISL recognition systems. The test accuracy achieved through this research is **89.50%**, which is **2.1%** improved over existing models [5]. The findings corroborate the potential of using the state-of-the-art tracking frameworks with deep learning techniques for the development of efficient and accurate ISL recognition systems [6]. This work increases academic understanding about the recognition of ISL and has practical implications for the development of assistive technologies that bridge communication gaps in various social contexts.

Index Terms

Sign language, assistive technology, deep learning, GRU, LSTM,

I. Introduction

A. Motivation

The aim is to develop comprehensive sign language recognition systems that enhance the inclusion of the hearing-impaired by improving communication. Available technology has not really had an influence on breaking barriers for the deaf community, who is majorly curtailed from accessing information and participating fully in daily life. Traditional methods of communication, like written text or lip reading, are grossly deficient in addressing key communication needs among the deaf community.

Current sign language recognition and translation systems mistranslate continuous sign languages because of the complexity of movements and handshapes. This would have numerous applications if accomplished; however, the development of such a system is challenged by the inadequacy of a standard dataset for this purpose.

This research is conducted in view of the improvement of continuous sign language recognition and translation using the INCLUDE [10] dataset. High-accuracy gesture recognition in real-time by advanced computer vision techniques and deep learning models for translation would be specifically employed in this project.

This research on use of assistive technology for persons with Hearing Impairment aims at enhancing the quality of life and their interaction within society. Enlarging the dataset and methodology in this domain will have a multiplier impact by increasing further research, innovation, and developments around recognition to act as enablers of better communication using sign languages among the deaf.

B. Contribution

This paper seeks primarily to evaluate the efficiency of a number of deep learning models, such as GRU, LSTM, and hybrid LSTM-GRU in designing and assessing a sign language recognition system using MediaPipe Hands and Pose for feature extraction. There are specific objectives guiding this research in meeting these goals, which are:

1) **Feature Extraction**: Keypoint coordinates can be extracted from video frames of ISL gestures by using MediaPipe Hands and Pose. Extracted coordinates will form the input features to the trained classification models.
2) **Model Design and Training**: Consider three independent deep learning models, LSTM, GRU, and hybrid LSTM-GRU, and train these models on the extracted features.
3) **Performance Evaluation**: Checking the performance of the created models on the INCLUDE dataset with accuracy and weighted F1 score as metrics.
4) **Comparison and Analysis**: A comparison of the performance of these three models, LSTM, GRU, and hybrid LSTM-GRU will be done. This analysis is necessary to find which one will end up being more powerful in recognizing ISL.
5) **Conclusion and Future Work**: The practical implications of the research findings will have to be discussed; possible real-life real-world applications are suggested, such as educational tools, assistive technologies, and communication aids for the deaf-mute community.

II. Dataset

The Indian Sign Language Dataset INCLUDE is a pretty recent effort at increasing the understanding and recognition of Indian Sign Language (ISL). This includes 4292 high-definition videos spanning 15 categories and 263 classes of various signs recorded by native signers, guaranteeing accuracy and precision in gesture representation. Each video is a recording of 1 ISL sign, signed by deaf students from St. Louis School for the Deaf, Adyar, Chennai.

A. Findings of INCLUDE dataset

After downloading and going through the dataset, we summarized its key features. We also found a few differences from the description given in the paper [10] and in the website [1].

It was observed that there were 4284 videos spread over 262 classes. There are roughly 16 videos in each class on average, and every video has a resolution of 1920x1080. Every video has an average duration of approximately 2.57 seconds. The videos have a 25 frames per second frame rate. In 16 classes, there was a folder named 'extra' amounting to 27 videos, which were discarded in this research. The website [1] also mentions a file about the train-test split, but that file is no longer available.

B. Class Distribution

There are 262 classes in the INCLUDE [10] dataset, each representing a unique ISL word. These classes have an unequal distribution of examples, with some classes having substantially more examples than others. A class has only four videos, while some classes have a maximum of 22 videos. The distribution of videos is shown as a graph plot in the Figure 1

C. Data Augmentation

We select cropping as our primary data augmentation approach to reduce class imbalance, increase the number of samples available, and improve the model's generalization. We observed that cropping at least 50 pixels from a video gives a variation in critical points. **FFmpeg** is the tool used to accomplish the cropping task. While cropping, make sure that the signer stays inside the video frame borders. After augmentation, the total number of videos amounted to 5371. The distribution of videos after augmentation is shown as a graph plot in the Figure 2

III. Methodology and experimental setup

The methodology chapter outlines the systematic approach taken in this study to design and assess an ISL recognition system. The approach combined the use of LSTM, GRU, and hybrid LSTM-GRU networks with MediaPipe Hands [12] and Pose [2] for feature extraction. The model architecture and design, the feature extraction procedure, the protocols for training and evaluating the models, and the specifics of implementation are all covered in this chapter.

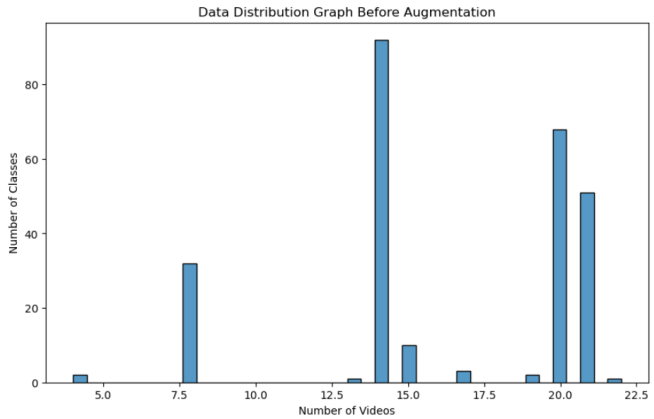

Figure 1: Data Distribution Graph of INCLUDE

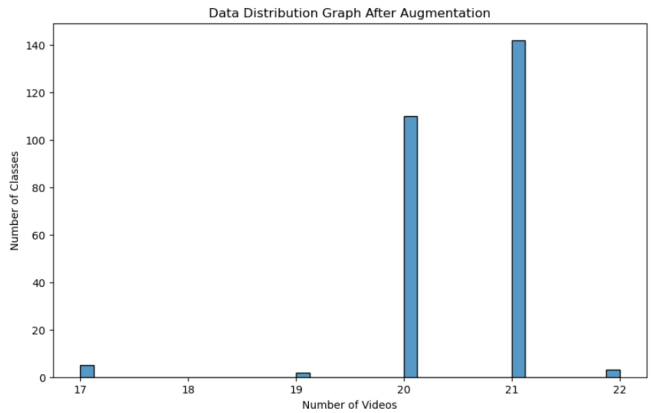

Figure 2: Data Distribution after augmentation

A. *Feature Extraction Using MediaPipe*

MediaPipe [7], created by Google, is an open-source framework that offers adaptable machine-learning solutions for live and streaming media across different platforms. Notably, MediaPipe Hands and MediaPipe Pose excel in real-time hand and body pose estimation. These tools empower developers to capture critical points from video feeds, which are crucial for applications like augmented reality, gesture recognition, and human-computer interaction.

Keypoint extraction in vision and machine learning was very important for understanding human movement by locating certain body areas, like joints or fingertips, to interpret gestures and poses. Such applications are relevant for tasks that include quick responses related to virtual reality, gaming, and robotics. The functionality of such applications relies a lot on the accuracy of its keypoint extraction.

Combining both keypoints from hands and poses generally gives more insight and one view on the activities of humans. In this regard, the changes in hand movement and the body pose can be returned in one dataset by means of combining these key points into a single feature vector, an approach which only improves the integrated one for various applications like sign language interpretation and tracking motion systems accurately and with better usability. This will involve the concatenation of key points from MediaPipe Hands and MediaPipe Pose into a 225-point data feature vector.

1) MediaPipe Hands: MediaPipe Hands [12] is a robust machine learning solution designed for the identification and tracking of hand gestures. It sets a good base for capturing key points from images of hands, making them very useful in applications ranging from augmented reality to even sign language translation.

a) Keypoints Extraction Process: MediaPipe Hands key point extraction involves detecting hand regions using a palm detection model and then identifying 21 key points on the hand with a landmark model. This

process optimizes accuracy and efficiency.Fig 3 shows the 21 keypoints of palm.

Provided with a large dataset of annotated hand images, the hand landmark model would be able to use a CNN in predicting the precise positions of key points against intricate backgrounds. In this case, the result will be 3D coordinates for each and every key point.

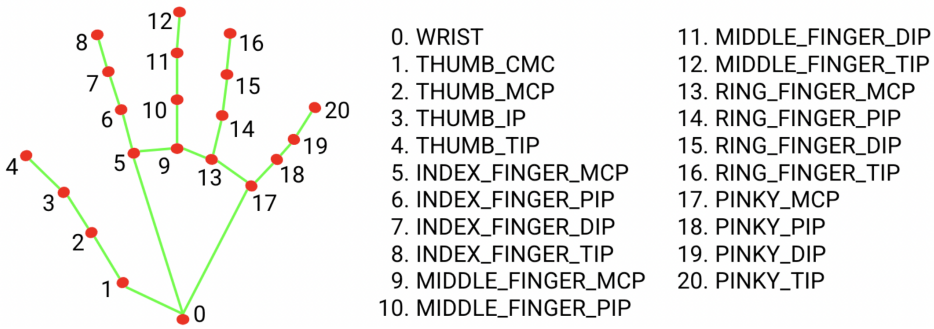

Figure 3: Hand Landmarks

b) Challenges in Hand Keypoints Extraction: Even with state-of-the-art capabilities, extracting hand key points does not come that easy in MediaPipe Hands. There might be several common issues related to this, such as, **Occlusion**: when there is overlap among hands or with other objects partially occluding the hands, it often results in the loss of keypoints data and reduced accuracy in their tracking. **Complex Backgrounds**: Cluttered or variable backgrounds, leading to mis-detections. **Variability in Hand Poses**: Here, the great variety of possible hand positions and the hand gestures serve to further headline the problem of correctly extracting the critical points.

2) MediaPipe Pose: The MediaPipe Pose library provides single-session tracking of the human body pose together with key point extractions for use cases such as fitness monitoring and animation.

a) Keypoints Extraction Process: MediaPipe Pose model picks up key points on the human body as shown in Fig 4, relating to shoulders, elbows, wrists, hips, knees, and ankles in real-time using MediaPipe Pose. This information can then be used in a detailed analysis of human postures and movements.

The key points are filtered for noise and interpolated where necessary, allowing the acquisition of refined key points for videos, which ensures preciseness and hence consistency and holds good enough for any type of analysis or application development.

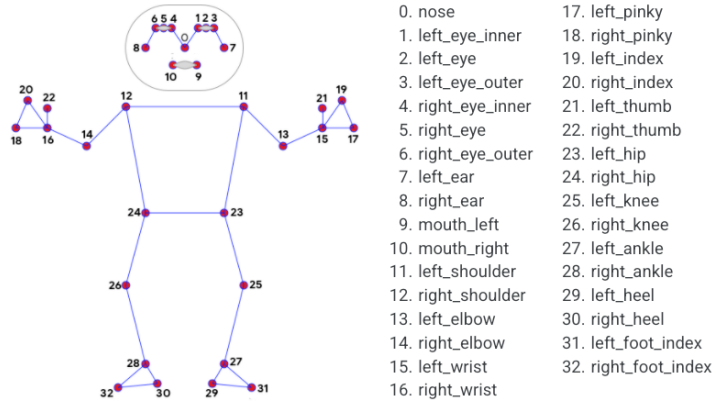

Figure 4: Pose Landmarks

b) Challenges in Hand Keypoints Extraction: The issues that may significantly affect the accuracy and reliability of MediaPipe Pose in extracting key points are, **Occlusion**: If one body part is hiding behind another, there might be difficulty for the model to exactly detect and map the key points hidden behind it, mostly happening in activity-cumbersome poses or during multiple people interactions. **Environmental Factors**: Noise and key point location inaccuracies can result from insufficient lighting, cluttered background, etc. Optimality in the environment is very much necessary to be ensured for accurate pose estimation. **Variability**

in Body Types and Clothing: Body shapes and body dimensions, along with the variety of clothing worn, would further affect how far the model generalizes at large about the demographics. Low-neck dresses, with tight clothes and unique body ratios, may cause problems.

3) Combining Keypoints to Form a Feature Vector: We will extract key points using MediaPipe Hands and MediaPipe Pose libraries to form a feature vector. MediaPipe Hands basically provides 21 key points for each hand, hence making 42 key points for both hands. While, MediaPipe Pose provides a total of 33 key points that cover the whole body. The overall result from these key points would be 75 (42 + 33), and this constitutes the base of our feature vector.

Data Extraction: For every frame in the video or for an image, the length, width, and depth coordinates of all keypoints shall be taken out, hence resulting in three values assigned to each keypoint.

Vector Assembly: For every frame, combine the x, y, and z coordinates of every key point in sequence to create an extended vector. With a total of 75 key points, each having three coordinates, the resultant feature vector will consist of 225 data points (75 key points * 3 coordinates).

Handling Missing Data: At times, key points might remain undetected because of occlusion or low image quality. Dealing with these absent values is crucial, whether through interpolation, referencing previous frame data, or assigning them a predetermined 'neutral' position to ensure the vector length consistency is maintained. In our case, we marked missing data as zeros.

B. Pre Processing and Saving Data

As mentioned in section II-A, the average length of videos is around 2.5 seconds, and the frame rate is 25fps. Based on this information, we decided to select 40 frames per video as time steps for feature extraction. The process is elaborated in the following steps :

- Using openCV, 40 frames were extracted from each of the 5371 videos, and key points were extracted from each frame using the method discussed in the previous section.
- The feature vector of dimension 40 x 225 for each video was saved as a numpy file
- After that, it was found that a total of 37 videos had frames less than 40 in them. These videos were dropped from the dataset.
- Finally, the dataset of 5334 videos with a feature vector of dimension 40 x 225 was saved in pickle format along with its label map.

C. Model Architecture

In this work, three distinct deep learning models—LSTM, GRU, and a hybrid LSTM-GRU network are developed and assessed. Each model is specifically designed to capture the temporal dynamics of input feature sequences.

a) Proposed hybrid LSTM-GRU: The proposed hybrid model combines the strengths of both LSTM and GRU cells to capture complex temporal dependencies in the input data. In this paper, the performance of the proposed model is compared with 3 LSTM and 3 GRU architectures followed by a comparison with existing technologies.

Architecture: In this proposed hybrid model, the output of the LSTM layer is connected to a GRU layer, followed by dense layers. This configuration leverages the complementary advantages of both LSTM and GRU units. The activation function **'ReLu'** is used in both layers. The final dense layer outputs class probabilities using a **softmax activation** function.

D. Training and Evaluation

The models are trained and evaluated on the INCLUDE dataset using a standardized training protocol to ensure fair comparisons.

1) Data Preparation: The dataset stored in pickle format in section III-B is loaded. The **class imbalance** is already resolved in section II-C

Train-Test Split: The loaded dataset is split into training set, validation set, and test set. The split is performed randomly, but it ensures that the distribution of classes remains consistent across sets. To perform this split, the "train-test-split" method of scikit-learn.the model-selection library is used.

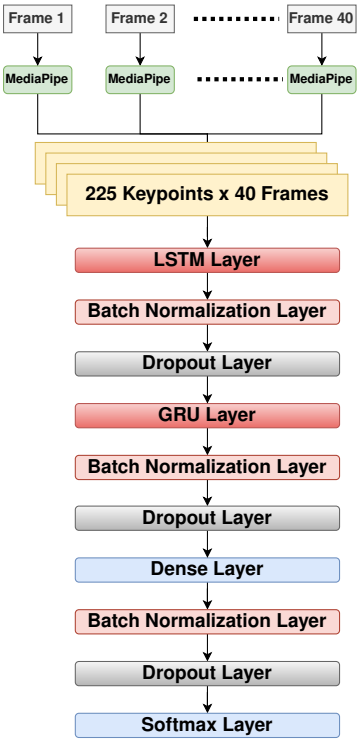

Figure 5: Hybrid LSTM-GRU model Architecture

2) Training Process: Hyperparameters, including learning rate, dropout rate, and units per layer, are tuned using the validation set. **Keras Tuner** technique is employed for hyperparameter optimization.

The **categorical cross-entropy loss function** is used for training, as it is well-suited for multi-class classification tasks.

Adam optimizer is used to adjust the learning rate dynamically during training, facilitating faster convergence.

a) Hyperparameter Tuning with Keras tuner: Another very important part associated with optimizing the performance of deep learning models in this research is the process of tuning the hyperparameters. Keras Tuner is used in this research to find out the optimum combination of hyperparameters for the classification models in a systematic way. Random Search, Hyperband, and Bayesian Optimization are some of the search algorithms supported by a practical and adaptable hyperparameter optimization framework offered by Keras Tuner.

b) Hyperparameters Considered: In this work, a variety of important hyperparameters are considered for tuning: **Units per Layer**: Number of units (neurons) in every layer influences model capacity to learn complex patterns. **Learning Rate**: This is the step size for the optimizer; it controls how quickly the model adapts during training. **Dropout Rate**: The proportion of units that shall be dropped during training to prevent overfitting.

The search space using the above method was **254 different configurations**. By enabling a thorough search across many hyperparameter configurations, hyperparameter tuning with Keras Tuner guarantees that the final model is well-optimized. This work finds the best configurations for LSTM, GRU, and hybrid LSTM-GRU models by methodically exploring the hyperparameter space, which improves performance on the INCLUDE dataset.

3) Evaluation Metrics: Several metrics are used to evaluate the performance of the models on the INCLUDE dataset. These measures offer a thorough evaluation of the models in terms of their overall performance, accuracy, and precision. Below are the details of each metric, along with the corresponding formulae:

a) Accuracy: The percentage of correctly identified examples relative to the total number of examples is known as accuracy. It offers a broad gauge of the model's effectiveness.

$$\text{Accuracy} = \frac{TN + TP}{TN + TP + FN + FP} \tag{1}$$

Where:

- TN is correctly predicted negative values.
- TP is correctly predicted positive values.
- FN is wrongly predicted negative values.
- FP is wrongly predicted positive values.

b) Precision: The precision metric quantifies the percentage of accurately categorized positive examples among all positive examples. It sheds light on how well the model avoids false positives.

$$\text{Precision} = \frac{TP}{FP + TP} \tag{2}$$

c) Weight Averaged F1-Score: The weighted F1 score uses a weight based on the number of true instances for each class when averaging the F1 scores, which are calculated independently for each class. This method provides a more realistic metric by accounting for class imbalance.

d) Confusion Matrix: The confusion matrix is a table that displays the true vs predicted class labels and provides an overview of a classification model's performance. It offers a thorough analysis of the model's effectiveness in various classes.

Table I: Confusion Matrix

Actual / Predicted	**Positive**	**Negative**
Positive	TP	FN
Negative	FP	TN

Using these metrics, the study systematically evaluates and compares the performance of LSTM, GRU, and proposed model, providing a robust understanding of their strengths and limitations in recognizing Indian Sign Language gestures.

E. Implementation Details

The implementation of the feature extraction and model training pipeline involves several technical components:

- **Programming Languages and Libraries**: Python is used for the implementation, which makes use of libraries like MediaPipe, Keras, Scikit-Learn, and TensorFlow. Real-time feature extraction is done using MediaPipe, while deep learning models are designed and trained using TensorFlow and Keras. Scikit-Learn is used to evaluate the model.
- **Hardware**: The training and evaluation are performed on a high-performance Desktop to accelerate the computation.
  - Processor is 12th Gen Intel(R) Core i7-12700 clocked at 2.10 GHz
  - Installed RAM is 16.0 GB (15.7 GB usable)
  - System type 64-bit operating system, x64-based processor
  - Integrated 8 GB Intel UHD Graphics 770

  Cloud-based solutions like Google Colab or dedicated GPU servers can be used to handle the intensive computational requirements.

IV. Results

An in-depth examination of the LSTM, GRU, and hybrid LSTM-GRU models' performance on the INCLUDE dataset is provided in this chapter. Weight Averaged F1-Score, accuracy, precision, and confusion matrix are the measures that form the basis of the previously stated evaluation. To give an all-encompassing picture of the model performance, we also incorporate visual representations of the training process, such as accuracy and loss graphs.

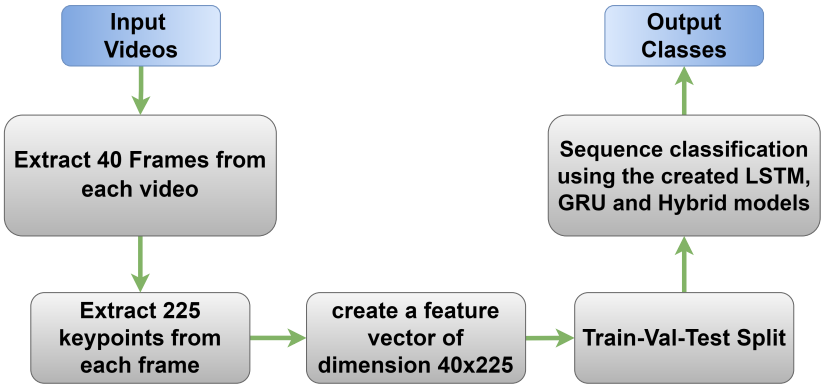

Figure 6: Flowchart of process of implementation

A. *Model Performance*

In this study, seven different model architectures were trained, tested, and evaluated.

Our proposed model architecture was able to achieve **89.50%** accuracy

Our proposed model architecture has one LSTM input layer and one hidden GRU layer, with batch normalization and dropout, connected to a dense layer connected to the softmax output layer. After hyperparameter tuning, the best parameters were selected, as shown in Fig. 7.

```
{'units_1': 64,
 'dropout_1': 0.2,
 'units_2': 160,
 'dropout_2': 0.4,
 'dense_units': 224,
 'dropout_3': 0.30000000000000004,
 'learning_rate': 0.001,
 'tuner/epochs': 100,
 'tuner/initial_epoch': 34,
 'tuner/bracket': 4,
 'tuner/round': 4,
 'tuner/trial_id': '0145'}
```

Figure 7: Best Parameters

Using these parameters, the proposed model was rebuilt, and its summary is a scene in Fig. 8

Proposed model was then fit with training and validation data prepared in section III-B and the training process is represented by Fig. 9 and Fig. 10

After training, the model was evaluated, and it can be seen in Fig. 11 and Fig. 12

B. *Comparison*

Since hyperparameter tuning is used to draw out the best possible result from a model, the above finding can be summarized as a relation between number of layers and model accuracy. Table II gives a summarized view of the findings of this study.

It is clear from Table II that all the models with two layers have achieved higher test accuracy. On top of that, it is clear that the Hybrid LSTM-GRU model has achieved the best accuracy of **89.50%** followed by two-layered GRU models with 89.03%. The gap between GRU-based models and LSTM-based models is evident from the table.

Table III shows the accuracy comparison between the proposed model and benchmark on the INCLUDE dataset.

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 40, 64)	74,240
batch_normalization (BatchNormalization)	(None, 40, 64)	256
dropout (Dropout)	(None, 40, 64)	0
gru (GRU)	(None, 160)	108,480
batch_normalization_1 (BatchNormalization)	(None, 160)	640
dropout_1 (Dropout)	(None, 160)	0
dense (Dense)	(None, 224)	36,064
batch_normalization_2 (BatchNormalization)	(None, 224)	896
dropout_2 (Dropout)	(None, 224)	0
dense_1 (Dense)	(None, 262)	58,950

Total params: 279,526 (1.07 MB)

Trainable params: 278,630 (1.06 MB)

Non-trainable params: 896 (3.50 KB)

Figure 8: Model Summary

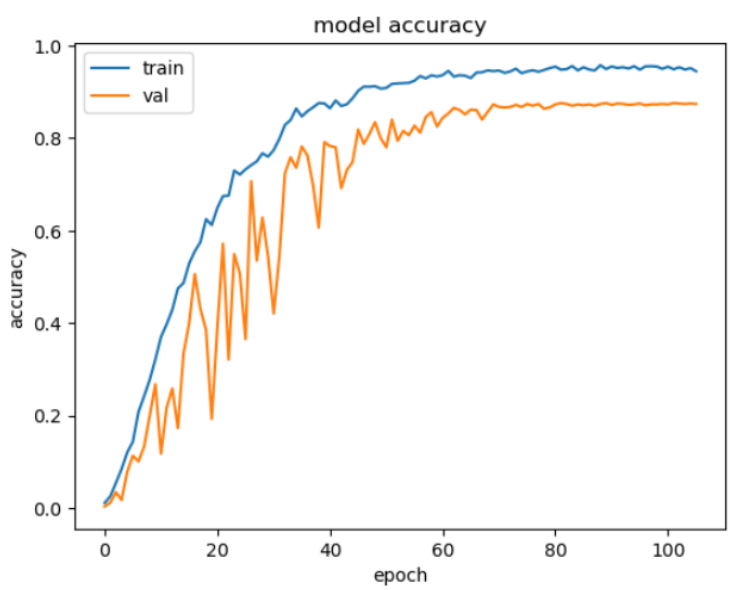

Figure 9: Accuracy during training

C. Comparative Analysis

The Comparison demonstrates that our proposed hybrid LSTM-GRU model outperforms the existing state-of-the-art methods in ISL recognition. Key observations include:

- **Higher Accuracy**: Our hybrid LSTM-GRU model achieves an accuracy of 89.50%, which is significantly higher than the reported accuracy of 87.4% by G Khartheesvar (2023) [5].
- **Improved F1-Score**: The F1-Score of 89.56% for our hybrid model indicates superior performance in handling precision and recall compared to other methods.

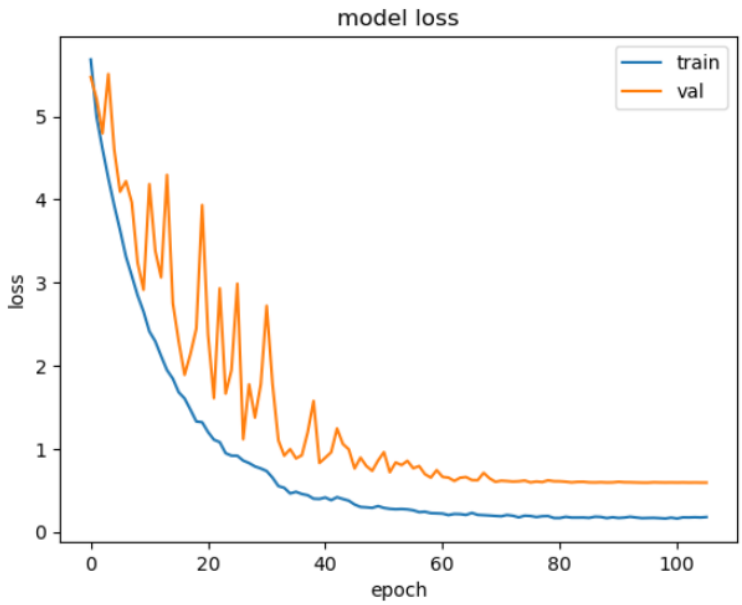

Figure 10: Loss during training

```
accuracy = metrics.accuracy_score(test_labels, predictions)
print(f'Accuracy = {accuracy}')
```

```
Accuracy = 0.895032802249297
```

```
precision = metrics.precision_score(test_labels, predictions, average='weighted')
print(f'Precision = {precision}')
```

```
Precision = 0.9229682688445575
```

```
f1score = metrics.f1_score(test_labels, predictions, average='weighted')
print(f'F1 Score = {f1score}')
```

```
F1 Score = 0.8956533799056163
```

Figure 11: Evaluation of model

- **Efficiency of Hybrid Approach**: Integrating LSTM and GRU units in a hybrid architecture leverages the strengths of both models, leading to enhanced performance metrics across the board.

D. Comparison with Existing Techniques

We compare our results with studies utilizing similar types and volumes of video datasets due to the limited research conducted on the relatively new INCLUDE dataset and the lack of other freely available ISL datasets. These comparable datasets share similar video and sign complexities and the number of classes to be classified, making them appropriate benchmarks. Table IV compares the proposed model in this study and existing techniques. As the table demonstrates, our suggested approach outperforms previous methods on comparable datasets.

V. Conclusions

This study proposes a method for recognizing isolated sign words in ISL. We employ MediaPipe hands and pose libraries to extract hand and pose landmark coordinates from video frames, which are converted into feature vectors and then utilized to train the hybrid LSTM-GRU model. By using cropping as a data

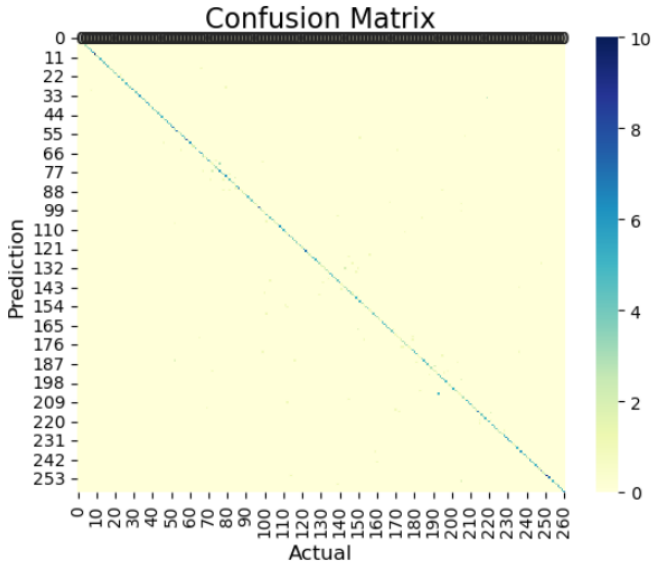

Figure 12: Confusion Matrix of model

Table II: Comparison of results from different models

Type	Method	Layers	Accuracy (%)
LSTM	Model 1	1	82.94
LSTM	Model 2	2	**84.72**
LSTM	Model 3	3	83.22
GRU	Model 4	1	83.50
GRU	Model 5	2	**89.03**
GRU	Model 6	3	88.47
Hybrid	Model 7	LSTM + GRU	**89.50**

augmentation method, we successfully handled class imbalance and increased our training data simultaneously. The model proposed in this study was evaluated on the INCLUDE dataset (89.50% accuracy, 89.56% F1 score). It was demonstrated that the proposed method could not correctly classify words when a word could have varied signs or when a signer performed it slightly differently.

More extensive ISL datasets may be employed to confirm the suggested method's resilience. Transformers, a more modern and effective deep learning-based model, could be utilized to boost the suggested method's efficiency. Lastly, techniques for identifying words in sign language from continuously recorded sign language films may be improved.

Acknowledgment

Kumar Navendu and Vineet Sahula thank project EICT Academy MNIT Jaipur for technical support to the work.

References

[1] "Include dataset website," accessed on May 3, 2024. [Online]. Available: https://zenodo.org/records/4010759

[2] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. L. Zhu, F. Zhang, and M. Grundmann, "Blazepose: On-device real-time body pose tracking," *ArXiv*, vol. abs/2006.10204, 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:219793039

[3] J. Duan, S. Zhou, J. Wan, X. Guo, and S. Li, "Multi-modality fusion based on consensus-voting and 3d convolution for isolated gesture recognition," 11 2016.

[4] A. Elboushaki, R. Hannane, A. Karim, and L. Koutti, "Multid-cnn: A multi-dimensional feature learning approach based on deep convolutional networks for gesture recognition in rgb-d image sequences," *Expert Systems with Applications*, vol. 139, p. 112829, 07 2019.

[5] G. Khartheesvar, M. Kumar, A. K. Yadav, and D. Yadav, "Automatic indian sign language recognition using mediapipe holistic and lstm network," *Multimedia Tools and Applications*, vol. 83, no. 20, pp. 58 329–58 348, Jun 2024. [Online]. Available: https://doi.org/10.1007/s11042-023-17361-y

Table III: Comparison of Accuracy with benchmarks

Method	Augmentation	Accuracy (%)
OpenPose + MobileNetV2 + BiLSTM [10]	Yes	86.32 (on 263 classes)
Proposed	Yes	**89.5** (on 262 classes)

Table IV: Comparison with existing similar work

Techniques	used dataset	Classes	Accuracy
SSD + ResNet50 + ESHR + HP + LSTM [9]	ChaLearn isoGD [11]	249	86.32%
2SCVN - 3DDSN [3]	ChaLearn isoGD [11]	249	67.19%
Multi-view Motion Attention CNN [8]	self created (ISL)	200	84.95%
MultiD-CNN [4]	ChaLearn isoGD [11]	249	72.53%
MediaPipe + LSTM [5]	INCLUDE dataset	262	87.4%
Proposed	INCLUDE dataset	262	**89.5%**

[6] D. Kothadiya, C. Bhatt, K. Sapariya, K. Patel, A.-B. Gil-González, and J. M. Corchado, "Deepsign: Sign language detection and recognition using deep learning," *Electronics*, vol. 11, no. 11, 2022. [Online]. Available: https://www.mdpi.com/2079-9292/11/11/1780

[7] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "Mediapipe: A framework for perceiving and processing reality," in *Third Workshop on Computer Vision for AR/VR at IEEE Computer Vision and Pattern Recognition (CVPR) 2019*, 2019.

[8] S. M., M. Prasad, and P. Kishore, "Multi-view motion modelled deep attention networks (m2da-net) for video based sign language recognition," *Journal of Visual Communication and Image Representation*, vol. 78, p. 103161, 05 2021.

[9] R. Rastgoo, K. Kiani, and S. Escalera, "Video-based isolated hand sign language recognition using a deep cascaded model," *Multimedia Tools and Applications*, vol. 79, 08 2020.

[10] A. Sridhar, R. G. Ganesan, P. Kumar, and M. Khapra, "INCLUDE: A Large Scale Dataset for Indian Sign Language Recognition," Sep. 2020. [Online]. Available: https://doi.org/10.1145/3394171.3413528

[11] J. Wan, S. Li, Y. Zhao, S. Zhou, I. Guyon, and S. Escalera, "Chalearn looking at people rgb-d isolated and continuous datasets for gesture recognition," 06 2016, pp. 761–769.

[12] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "Mediapipe hands: On-device real-time hand tracking," *ArXiv*, vol. abs/2006.10214, 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:219792872